

ПРАКТИЧЕСКАЯ РАБОТА №6

«Алгоритмы поиска»

Часть 6.1. Быстрый доступ к данным с помощью хеш-таблиц

Вариант 4

Цель: освоить приёмы хеширования и эффективного поиска элементов множества.

Вариант 4. Метод хеширования — цепное хеширование (по согласованию с преподавателем вместо открытой адресации). Структура элемента — регистрация малого предприятия: номер лицензии, название, учредитель.

1. Формулировка задачи

Разработать приложение, использующее хеш-таблицу для прямого доступа к элементам динамического множества, реализованного на массиве. Класс содержит массив записей, хеш-таблицу и операции вставки, удаления, поиска по ключу, вывода. Реализовать расширение таблицы с рехешированием, автозаполнение 5–7 записями и текстовый командный интерфейс.

2. Математическая модель решения

Массив записей `records` — `vector<Enterprise>`, записи хранятся подряд без пропусков.

Хеш-таблица `table` — массив из m указателей на цепочки. Цепочка — однонаправленный список узлов `Node` (ключ, индекс записи в `records`, указатель на следующий узел); в цепочке с номером h хранятся все ключи с хешем h . Пустая цепочка — `nullptr`.

Хеш-функция — метод деления (начальное $m = 7$):

$$h(K) = K \bmod m$$

Операции:

- **поиск** — вычисляется $h(K)$, просматривается только цепочка `table[h]` до узла с ключом K (успех) или до конца цепочки (неудача); по индексу из узла запись берётся из `records`;
- **вставка** — если ключа нет, запись добавляется в конец `records`, а узел (ключ, индекс) — в начало цепочки `table[h]`; если цепочка не пуста, это коллизия;
- **удаление** — узел исключается из цепочки (если он первый, `table[h]` указывает на следующий узел, иначе предыдущий узел ссылается на следующий) и освобождается; на место удалённой записи в `records` переносится последняя запись, её индекс в цепочке обновляется;

- **рехеширование** — если после вставки коэффициент нагрузки $n/m > 0.75$ (n — число записей), размер таблицы увеличивается вдвое, и все ключи из `records` заново размещаются по цепочкам с новым $h(K) = K \bmod 2m$. Это сохраняет цепочки короткими, а поиск — в среднем $O(1 + n/m)$.

3. Код программы

Объявления (`table.h`):

```
1 #pragma once
2
3 #include <string>
4 #include <vector>
5
6 // Запись: регистрация малого предприятия. Ключ - номер лицензии.
7 struct Enterprise {
8     int license;           // номер лицензии
9     std::string name;      // название
10    std::string founder;   // учредитель
11 };
12
13 // Элемент цепочки (однонаправленного списка): ключ и индекс записи в массиве
14 struct Node {
15     int key;               // номер лицензии
16     int index;             // индекс записи в массиве records
17     Node* next;           // следующий элемент цепочки или nullptr
18 };
19
20 // Хеш-таблица с цепным хешированием
21 class EnterpriseTable {
22 public:
23     EnterpriseTable(int size = 7);
24     ~EnterpriseTable();
25     // копирование запрещено: две таблицы освободили бы одни и те же узлы
26     EnterpriseTable(const EnterpriseTable&) = delete;
27     EnterpriseTable& operator=(const EnterpriseTable&) = delete;
28
29     bool insert(const Enterprise& e);           // false, если ключ уже есть
30     bool remove(int license);                   // false, если ключа нет
31     const Enterprise* find(int license) const; // nullptr, если ключа нет
32     void print() const;
33
34 private:
35     std::vector<Enterprise> records; // массив полезных данных
36     std::vector<Node*> table;        // хеш-таблица: указатели на цепочки
37
38     int hash(int key) const;
39     Node* findNode(int key) const;
40     void pushFront(int key, int index);
41     void clear();
```

```

42     void rehash();
43 };
44

```

Реализация класса (table.cpp):

```

1  #include "table.h"
2
3  #include <cstdio>
4  #include <iostream>
5
6  EnterpriseTable::EnterpriseTable(int size) : table(size, nullptr) {}
7
8  EnterpriseTable::~~EnterpriseTable() {
9      clear();
10 }
11
12 // Хеш-функция методом деления:  $h(K) = K \bmod m$ 
13 int EnterpriseTable::hash(int key) const {
14     return key % (int)table.size();
15 }
16
17 // Возвращает узел с ключом key или nullptr.
18 // Просматривается только одна цепочка - с индексом h(key).
19 Node* EnterpriseTable::findNode(int key) const {
20     for (Node* p = table[hash(key)]; p != nullptr; p = p->next)
21         if (p->key == key) return p;
22     return nullptr;
23 }
24
25 // Добавляет пару (key, index) в начало цепочки с индексом h(key)
26 void EnterpriseTable::pushFront(int key, int index) {
27     int h = hash(key);
28     table[h] = new Node{key, index, table[h]};
29 }
30
31 // Освобождает память всех узлов, все цепочки становятся пустыми
32 void EnterpriseTable::clear() {
33     for (Node*& head : table) {
34         while (head != nullptr) {
35             Node* next = head->next;
36             delete head;
37             head = next;
38         }
39     }
40 }
41
42 // Увеличивает таблицу вдвое и заново раскладывает все ключи по цепочкам
43 void EnterpriseTable::rehash() {
44     int m = 2 * table.size();
45     std::cout << "Коэффициент нагрузки > 0.75 → рехеширование: m = "
46               << table.size() << " → " << m << '\n';
47     clear();

```

```

48     table.assign(m, nullptr);
49     for (int i = 0; i < (int)records.size(); i++)
50         pushFront(records[i].license, i);
51 }
52
53 bool EnterpriseTable::insert(const Enterprise& e) {
54     if (findNode(e.license) != nullptr) return false;
55     int h = hash(e.license);
56     std::cout << "Ключ " << e.license << ": h = " << h
57         << (table[h] ? " (коллизия, в начало цепочки)" : "") << '\n';
58     records.push_back(e);
59     pushFront(e.license, records.size() - 1);
60     if (records.size() > 0.75 * table.size()) rehash(); // n/m > 0.75
61     return true;
62 }
63
64 bool EnterpriseTable::remove(int license) {
65     // ищем узел в цепочке, запоминая предыдущий
66     int h = hash(license);
67     Node* prev = nullptr;
68     Node* cur = table[h];
69     while (cur != nullptr && cur->key != license) {
70         prev = cur;
71         cur = cur->next;
72     }
73     if (cur == nullptr) return false;
74
75     // исключаем узел из цепочки
76     if (prev == nullptr) table[h] = cur->next; // узел - начало цепочки
77     else prev->next = cur->next;
78     int idx = cur->index;
79     delete cur;
80
81     // на место удаленной записи переносим последнюю запись массива
82     int last = records.size() - 1;
83     if (idx != last) {
84         records[idx] = records[last];
85         findNode(records[idx].license)->index = idx;
86     }
87     records.pop_back();
88     return true;
89 }
90
91 const Enterprise* EnterpriseTable::find(int license) const {
92     Node* p = findNode(license);
93     return p == nullptr ? nullptr : &records[p->index];
94 }
95
96 void EnterpriseTable::print() const {
97     int m = table.size(), n = records.size();
98     std::printf("Хеш-таблица: m = %d, записей n = %d, n/m = %.2f\n", m, n,
99         (double)n / m);
100     for (int h = 0; h < m; h++) {

```

```

101         std::printf("%4d: ", h);
102         if (table[h] == nullptr) std::cout << "nil";
103         for (Node* p = table[h]; p != nullptr; p = p->next)
104             std::cout << p->key << '[' << p->index << ']'
105                 << (p->next ? " → " : "");
106         std::cout << '\n';
107     }
108     std::cout << "Массив записей:\n";
109     for (int i = 0; i < n; i++)
110         std::cout << " [" << i << "] " << records[i].license << " | "
111             << records[i].name << " | " << records[i].founder << '\n';
112 }
113

```

Командный интерфейс (main.cpp):

```

1  #include <iostream>
2  #include <string>
3
4  #include "table.h"
5
6  std::string readLine(const std::string& prompt) {
7      std::cout << prompt;
8      std::string s;
9      std::getline(std::cin, s);
10     return s;
11 }
12
13 int readInt(const std::string& prompt) {
14     return std::stoi(readLine(prompt));
15 }
16
17 int main() {
18     EnterpriseTable t;
19
20     // Автозаполнение. При m = 7: h(100002) = h(100009) = h(100016) = 0,
21     // h(100005) = h(100012) = 3 - есть коллизии.
22     std::cout << "Автозаполнение таблицы:\n";
23     t.insert({100002, "Ромашка", "Иванов И.И."});
24     t.insert({100005, "Василек", "Петров П.П."});
25     t.insert({100009, "Береза", "Сидоров С.С."});
26     t.insert({100012, "Ландыш", "Смирнова А.А."});
27     t.insert({100016, "Клевер", "Кузнецов Д.Н."});
28     t.print();
29
30     while (true) {
31         std::cout << "\n1 - вставка, 2 - удаление, 3 - поиск, 4 - вывод, "
32             << "0 - выход\n";
33         std::string cmd = readLine("> ");
34         if (!std::cin || cmd == "0") break;
35
36         if (cmd == "1") {
37             Enterprise e;

```

```

38         e.license = readInt("Номер лицензии: ");
39         e.name = readLine("Название: ");
40         e.founder = readLine("Учредитель: ");
41         if (!t.insert(e))
42             std::cout << "Ключ " << e.license << " уже есть\n";
43     } else if (cmd == "2") {
44         int key = readInt("Номер лицензии: ");
45         std::cout << (t.remove(key) ? "Удалено\n" : "Не найдено\n");
46     } else if (cmd == "3") {
47         int key = readInt("Номер лицензии: ");
48         const Enterprise* e = t.find(key);
49         if (e)
50             std::cout << "Найдено: " << e->license << " | " << e->name
51                 << " | " << e->founder << '\n';
52         else
53             std::cout << "Не найдено\n";
54     } else if (cmd == "4") {
55         t.print();
56     } else {
57         std::cout << "Неизвестная команда\n";
58     }
59 }
60 }
61

```

4. Результаты тестирования

Автозаполнение 5 записями ($m = 7$). Ключи 100002, 100009, 100016 имеют $h = 0$, ключи 100005, 100012 — $h = 3$; в квадратных скобках — индекс записи в `records`:

```

1  Ключ 100002: h = 0
2  Ключ 100005: h = 3
3  Ключ 100009: h = 0 (коллизия, в начало цепочки)
4  Ключ 100012: h = 3 (коллизия, в начало цепочки)
5  Ключ 100016: h = 0 (коллизия, в начало цепочки)
6  Хеш-таблица: m = 7, записей n = 5, n/m = 0.71
7      0: 100016[4] → 100009[2] → 100002[0]
8      1: nil
9      2: nil
10     3: 100012[3] → 100005[1]
11     4: nil
12     5: nil
13     6: nil

```

Таблица 4.1 — Тесты операций

№	Действие	Результат
1	Поиск 100002 (конец цепочки 0)	Найдено: Ромашка
2	Поиск 100023 ($h = 0$, ключа нет)	Просмотрена цепочка 0 — не найдено
3	Удаление 100009 (середина цепочки 0)	Цепочка 0: 100016[2] → 100002[0], запись 100016 перенесена в records[2]
4	Удаление 100016 (начало цепочки 0)	Цепочка 0: 100002[0]
5	Поиск 100002 после удалений	Найдено: Ромашка
6	Вставка 100030 ($h = 0$)	Коллизия, в начало цепочки 0
7	Вставка 100005 (дубликат)	«Ключ 100005 уже есть»
8	Удаление 999999	«Не найдено»
9	Вставка 100001 ($h = 6$)	$n/m = 5/7 = 0.71$ — без рехеширования
10	Вставка 100004 ($h = 2$)	$n/m = 6/7 = 0.86 > 0.75$ — рехеширование $m = 7 \rightarrow 14$

Таблица после теста 4 и после теста 10 (пустые цепочки не показаны):

1	$m = 7, n = 3$	$m = 14, n = 6$
2	0: 100002[0]	0: 100030[3] → 100002[0]
3	3: 100012[2] → 100005[1]	2: 100004[5]
4		3: 100005[1]
5		10: 100012[2]
6		13: 100001[4]

После рехеширования $h(K) = K \bmod 14$: цепочка 3 распалась (100012 перешёл в цепочку 10), нагрузка $6/14 = 0.43$.

5. Вывод

Реализован класс хеш-таблицы с цепным хешированием: ключи с одинаковым хешем хранятся в одной цепочке (однонаправленном списке), поэтому коллизии не мешают вставке, поиску и удалению. Тесты подтвердили корректность операций для ключей, вызвавших коллизии (в начале, середине и конце цепочки). Рехеширование с удвоением размера при $n/m > 0.75$ сохраняет цепочки короткими, а время операций — в среднем $O(1)$. Цель работы достигнута.